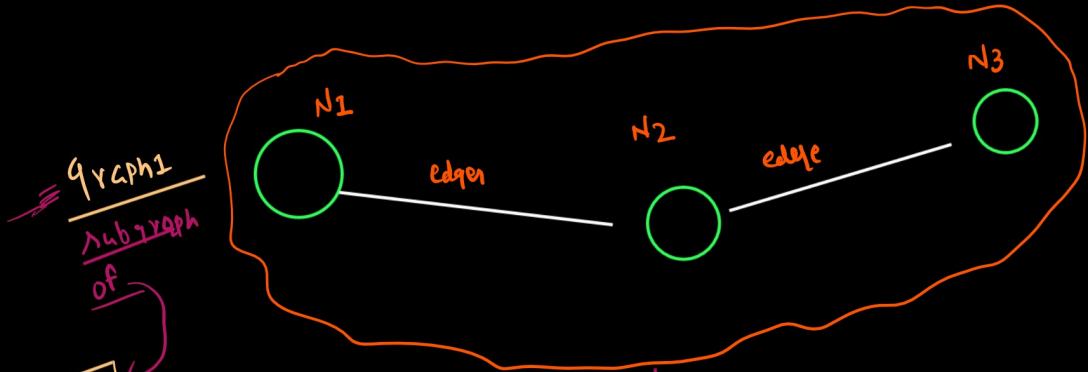
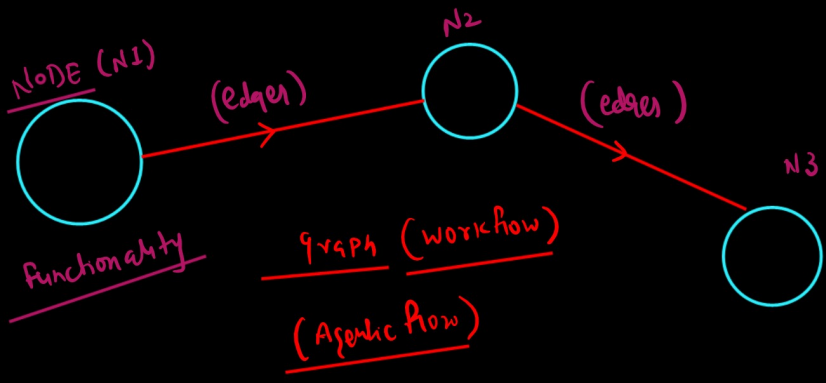
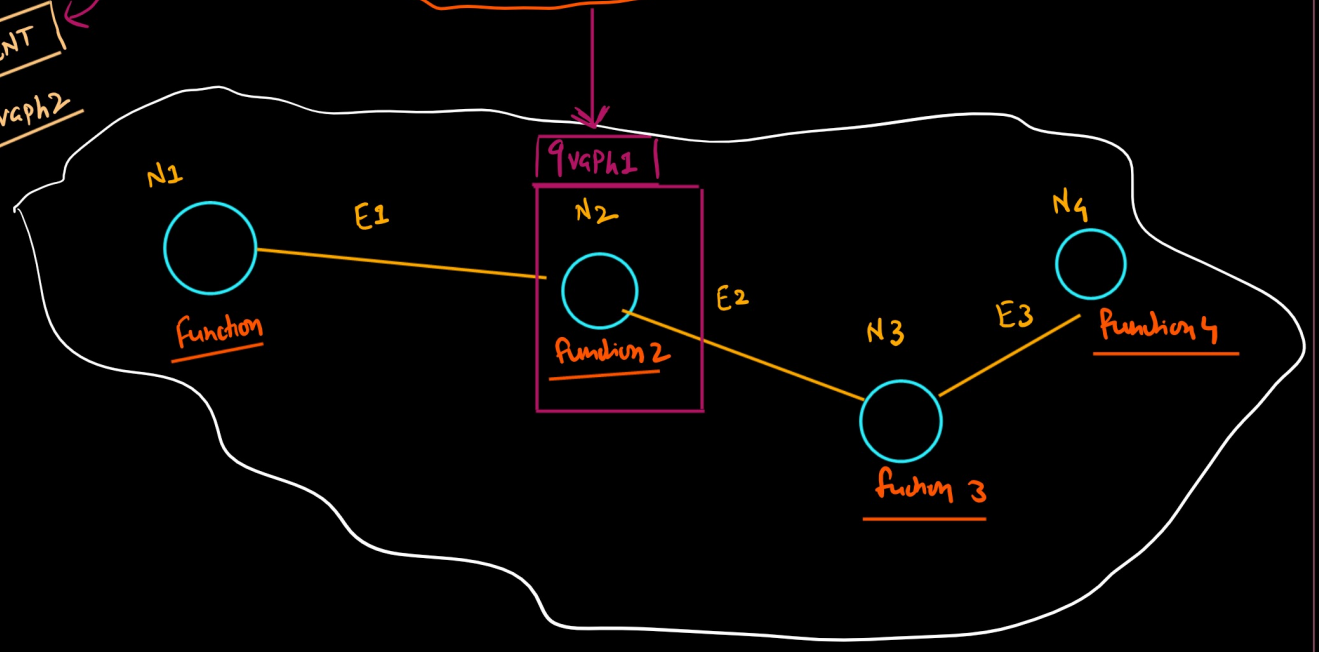


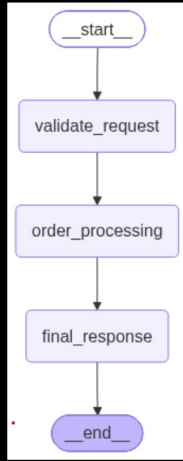
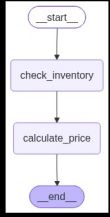
Subgraph → Graph → Collection of nodes/edge



PARENT
graph2



graph (Parent)

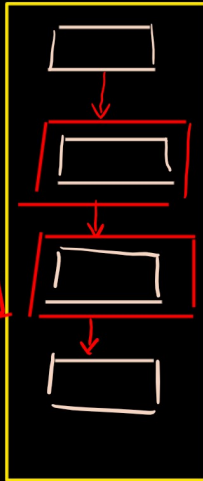


graph (subgraph)

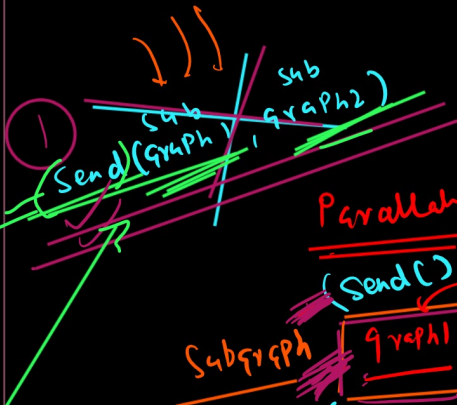
Parent graph

Subgraph 1

Sequentially



Subgraph 2



Parent graph

Parallel

Send()

Subgraph

Send(Graph1, ...)
Send(Graph1, ...)
Send(Graph1, ...)

Parallel

{
 Startgraph ()
 Intrap ()
 send ()
 Command ()
 }

FAN OUT

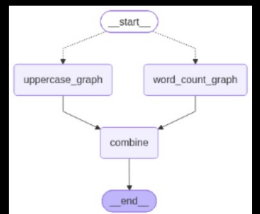
send()

graph 2

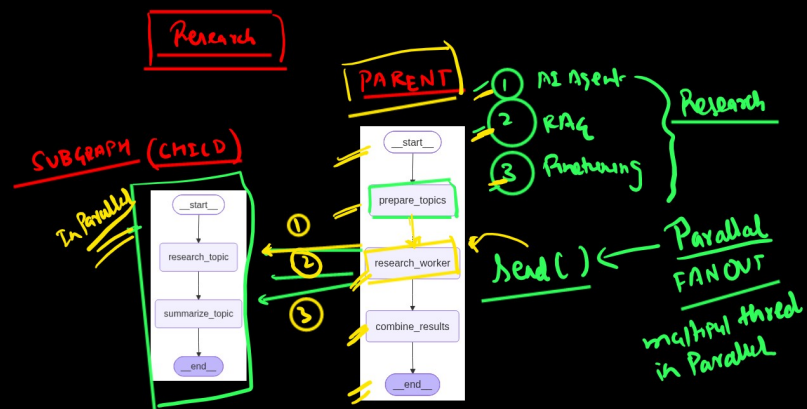
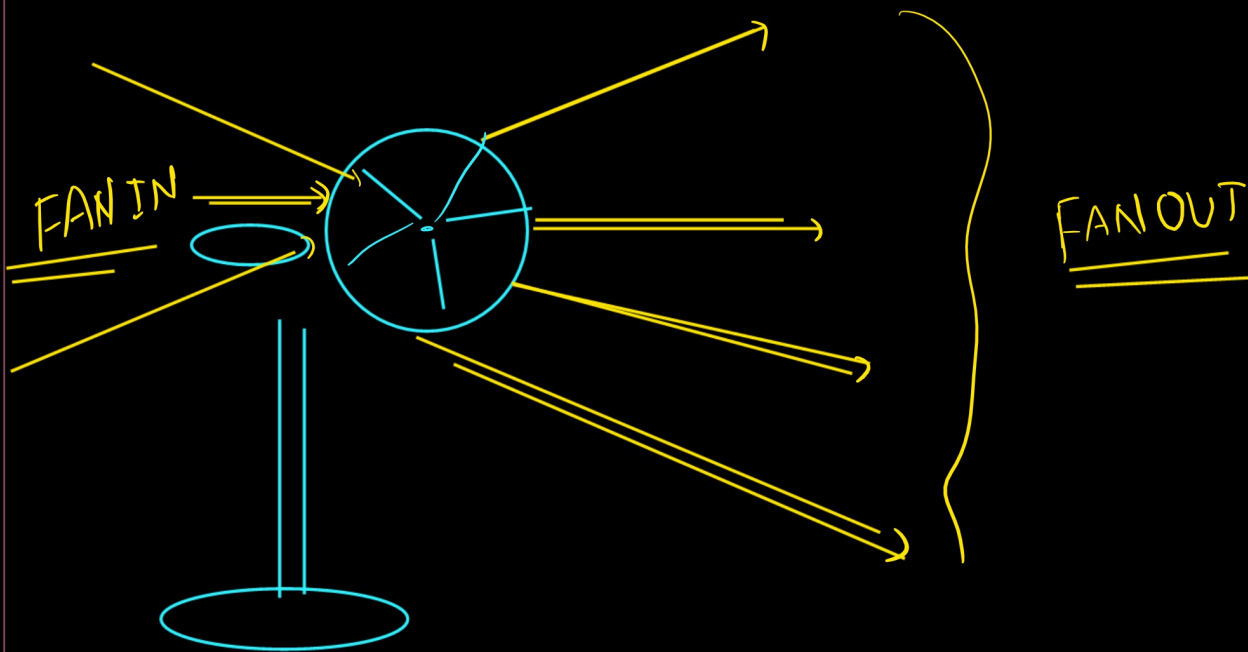
subgraph

Send(Graph2, ...)
Send(Graph2, ...)
Send(Graph2, ...)

Parallel



multi Agent system



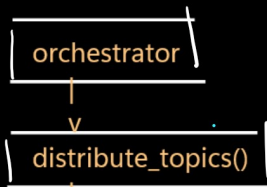
```
research_parent_builder.add_edge(
    START,
    "prepare_topics",
)
```

```
research_parent_builder.add_conditional_edges("prepare_topics",
    distribute_topics,
    "research_worker",
)
```

Send() OIP

```
research_parent_builder.add_edge(
    "research_worker",
    "combine_results",
)
```

```
research_parent_builder.add_edge(
    "combine_results",
    END,
)
```



```
|--- Send("worker", {"topic": "RAG"})  
|--- Send("worker", {"topic": "Agents"})  
|--- Send("worker", {"topic": "Memory"})  
v
```

worker
worker
worker

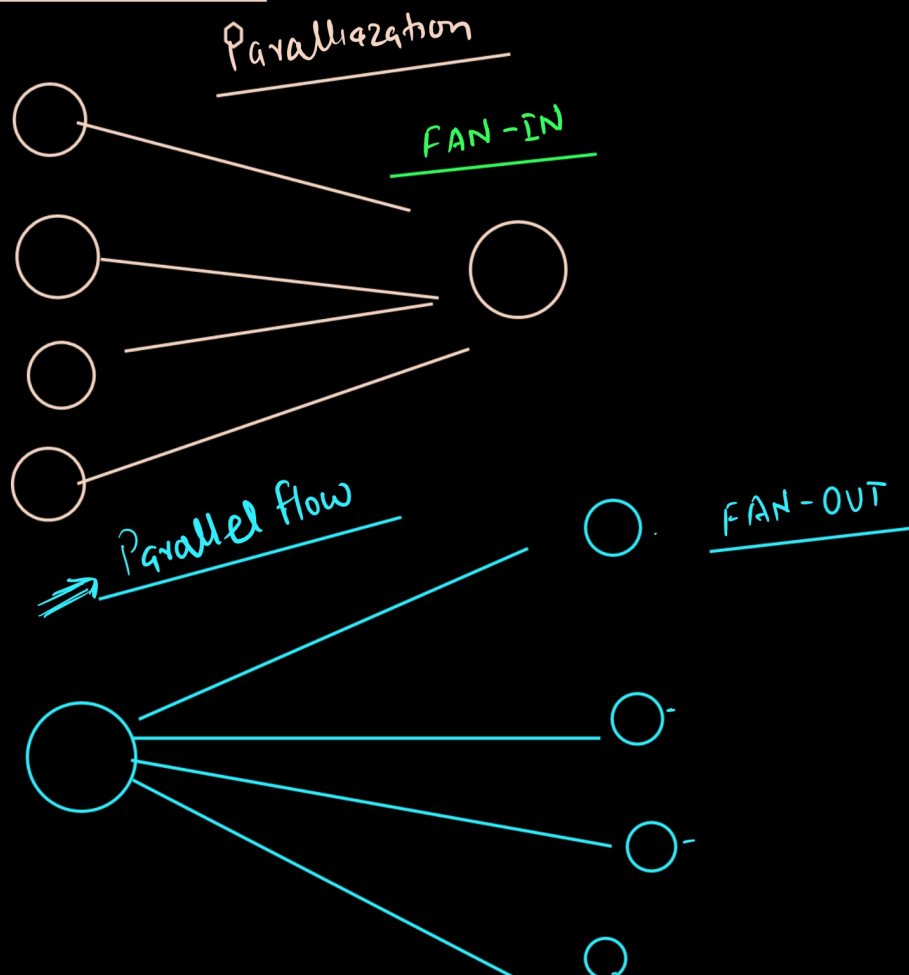
```
def distribute_topics(state):  
    return [  
        Send("worker", {"topic": topic})  
        for topic in state["topics"]  
    ]
```

```
builder.add_conditional_edges(  
    "orchestrator",  
    distribute_topics  
)
```

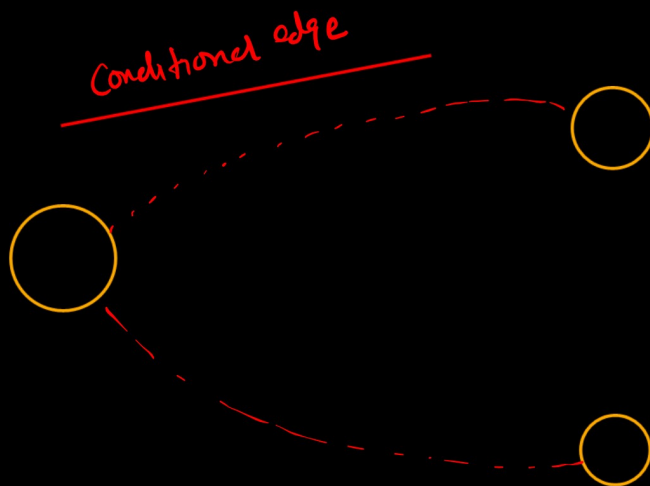
① Sequential workflow



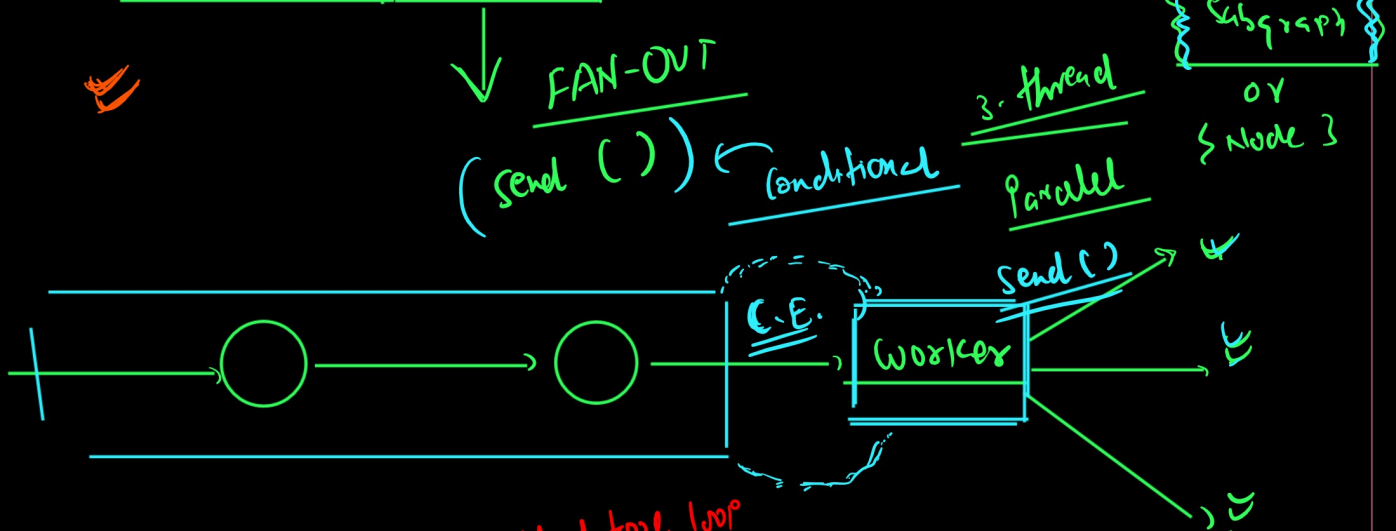
② Parallelization (parallel workflow)



③ Routing → Conditional Routing
dynamic branch

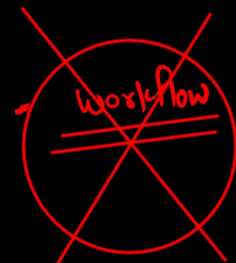
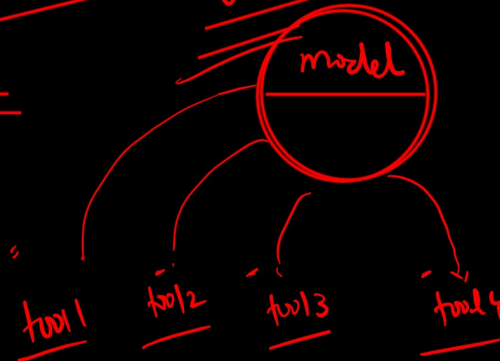


④ Orchestrator | Worker

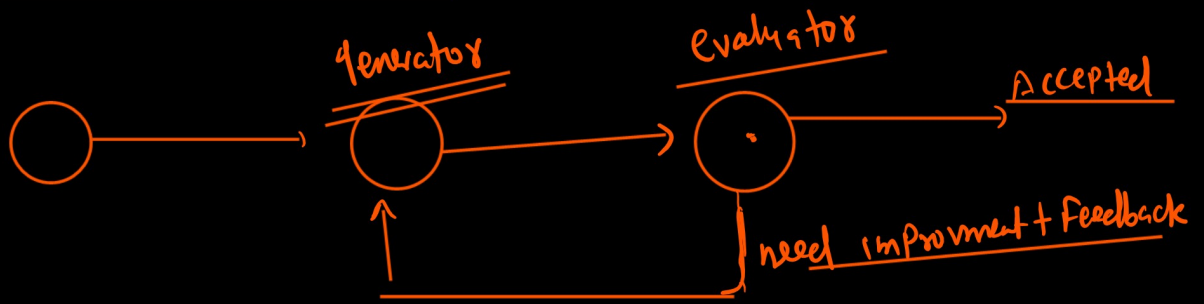


model controlled tool loop
dynamically

⑤ Agent loop



6 evaluator - optimizer (feedback loop)



Langgraph

building a graph

- 1 add-node()
- 2 add-edges()
- 3 add-conditional-edges()
- 4 add-sequence()
- 5 set-conditional-entry-point()
- 6 set-finish-point()
- 7 stategraph()

execution

graph.invoke()
 graph.stream()
 graph.getState()
 graph.getStateHistory()
 graph.updateState()

getConfig()
 getState()
 getRuntime()

Control-Flow

Send()

dynamic fan-out

Command()

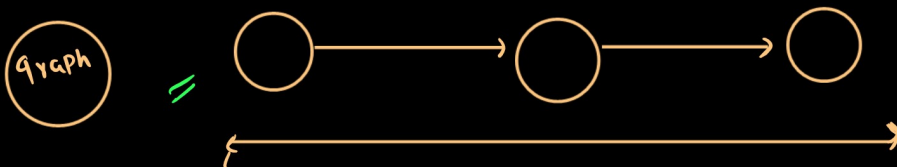
goto, update, resume

Interrupt()

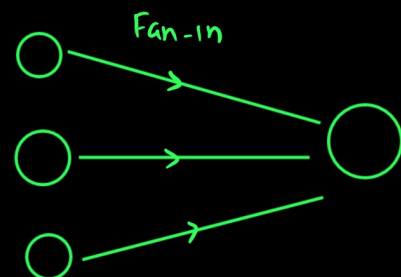
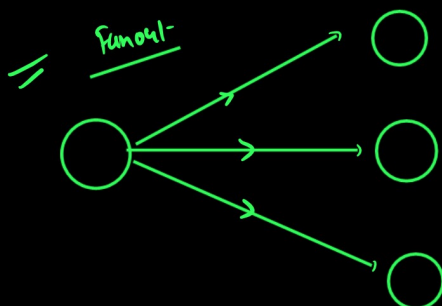
Pause workflow
 for external human
 input

Workflow

Sequential → we know step & order of those step

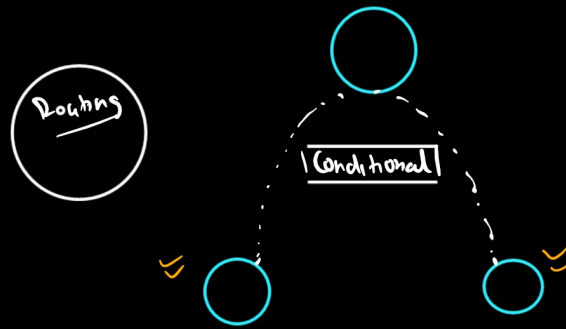


Parallel (fanout/fanin) → Several independent-jobs that should all run
 fixed Parallel flow



Routing (conditional based routing)

// I have to choose correct flow based on the condition

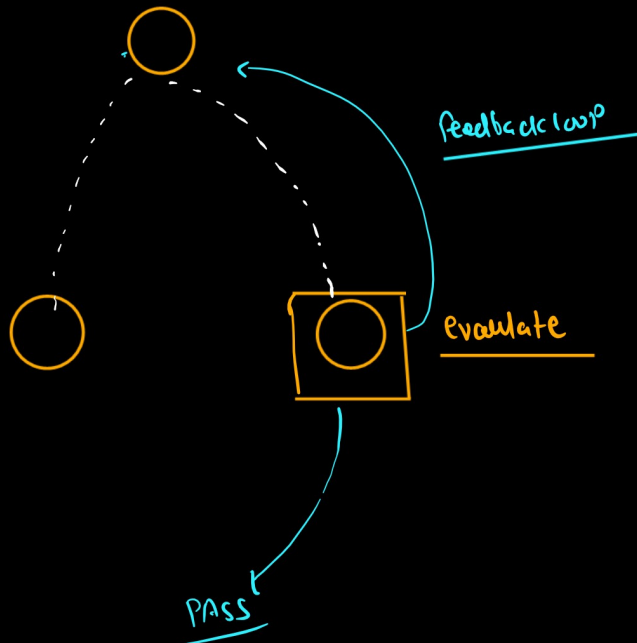


Feedback loop (evaluator-optimizer)

Loop-eng.

Loop-threshold

You will decide that based on req.



↓ ↓ ↓
Orchestrator - worker

(Send())
(Command())

// tool calling flow (dim)

(Orchestrator) Worker

